Google Developer Group
Universitas Sriwijaya

Learning 8

Side Effects & Data Fetching

useEffect to interact with external systems

Pure Rendering in React

React components should ideally be pure functions.

- UI is derived from state and props
- Same input → same output
- No external interaction during render
- Keeps components predictable

```
function Greeting({ name }) {
  return <h1>Hello {name}</h1>
}
```

What is a Side Effect?

A side effect is any operation that interacts with the outside world

- Fetching data from an API
- Updating the document title
- Subscribing to events
- Using browser APIs (localStorage, timers)

Important rule: Side effects should not run during render

React Solution: useEffect

React provides a hook for side effects.

- Runs code after render
- Used for external interactions
- Keeps rendering logic pure
- Can control when the effect runs

```
useEffect(() => {
  console.log("Component mounted")
}, [])
```

Common Side Effect Contexts

useEffect is used when React interacts with external systems.

Interacting with Browser APIs

- Runs code after render
- Used for external interactions
- Keeps rendering logic pure
- Can control when the effect runs

UI behavior

- Trigger animations
- Imperative DOM updates

Common Side Effect Contexts

Persisting state

- Sync state to localStorage

```
useEffect(() => {
  localStorage.setItem("theme", theme)
}, [theme])
```

Understanding Dependencies

The dependency array controls when an effect runs.

General rule:

- React reruns the effect when dependencies change

```
useEffect(() => {
  console.log("Count changed")
}, [count])
```

Effect runs when count changes

When Does useEffect Run?

There are three common patterns.

Run once (on mount)

Use cases:

- initial setup
- fetch data
- subscriptions

```
useEffect(() => {
  console.log("Component mounted")
}, [])
```

Run when a value changes

Use cases:

- sync state
- react to changes

```
useEffect(() => {
  console.log("User updated")
}, [user])
```

Run on every render

Use cases (rare):

- debugging
- measuring layout

```
useEffect(() => {
  console.log("Component rendered")
})
```

Mental Model for useEffect

Important principle: React separates rendering from side effects.

Rendering phase:

- React calculates the UI

Effect phase:

- useEffect runs external logic

This keeps components predictable and easier to debug.

Fetching Data in React

Fetching data is one of the most common side effects.

Typical flow:

- Component renders
- Request data from API
- Update state with response
- UI re-renders with data

Basic Data Fetching Pattern

Data fetching usually involves three states.

Typical flow:

- Loading
- Success (data)
- Error

```
const [data, setData] = useState([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)
```

Fetching Data with useEffect

Fetching should run after the component mounts.

Why?

- Prevents request during render
- Runs once when component loads

```
useEffect(() => {
  fetchData()
}, [])
```

API Request Best Practice

Basic async pattern with error handling.

Concepts:

- try / catch for error handling
- finally for loading state
- keep UI responsive

```
async function fetchData() {
  try {
    const res = await fetch(API_URL)
    const data = await res.json()
    setData(data)
  } catch (err) {
    setError(err)
  } finally {
    setLoading(false)
  }
}
```

Organizing API Code

Even without custom hooks, we can keep things clean.

Simple best practices:

- Store API base URL in .env
- Create a small api.js helper
- Separate fetching logic from UI when possible

```
export const API_URL =
  import.meta.env.VITE_API_URL
```

Key Takeaways

- Side effects interact with external systems
- useEffect runs after rendering
- Common uses: events, timers, API calls
- Data fetching requires loading and error state
- Keep rendering logic separate from async logic

Next step: Making reusable async logic with Custom Hooks

Challenge! (FE Member Only*)

Pelajari dan eksplorasi lebih mendalam terkait *useEffect*

1. Race Conditions
2. Dependency Array "Pitfalls"
3. Synchronization vs Event Handlers
4. UseEffect untuk Subscription

Selamat Mencoba ~

Challenge! (FE Member Only*)

1. Cari sumber public API di internet, misal https://openweathermap.org/api
2. Buat sebuah simple web yang melakukan fetching data dari API tersebut
3. Implementasikan apa yang telah dipelajari
4. Lakukan pull request ke Frontend Development GDGoC UNSRI 25/26 untuk memasukkan link repo github anda di folder /8-side-effect-fetch-api/task.txt

Selamat Mencoba ~

Google Developer Group
Universitas Sriwijaya

Thank You !